Федеральное государственное образовательное бюджетное учреждение
высшего образования
«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ РФ»
Департамент анализа данных и машинного обучения
Кафедра Искусственного Интеллекта

Курсовой проект по дисциплине «Машинное обучение» на тему:
«Разработка системы детекции и классификации шахматных фигур с использованием методов глубокого обучения»

Выполнил: студент группы ПМ23-1 факультета
информационных технологий и анализа
больших данных Кидысюк А.В.
Научный руководитель:
Андрианов А.В.

Москва 2025 г

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 3
Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ДЕТЕКЦИИ ОБЪЕКТОВ..... 5
1.1. Обзор современных методов компьютерного зрения (YOLO, SSD, Faster R-CNN)..... 5
1.2. Специфика формирования наборов данных для обучения..... 7
Глава 2. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПОДГОТОВКА ДАННЫХ..... 9
2.1. Постановка задачи и инструментарий..... 9
2.2. Генерация синтетического набора данных..... 10
2.3. Формирование набора реальных данных..... 12
Глава 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ МОДЕЛЕЙ..... 15
3.1. Обучение и тестирование модели YOLOv11..... 15
3.2. Анализ эффективности модели SSD..... 20
3.3. Исследование архитектуры Faster R-CNN..... 22
3.4. Сравнительный анализ результатов..... 24
ЗАКЛЮЧЕНИЕ..... 26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ..... 28
ПРИЛОЖЕНИЕ А. Код генерации синтетических данных..... 30
ПРИЛОЖЕНИЕ Б. Код захвата и разметки реальных данных..... 32

ВВЕДЕНИЕ

Актуальность темы исследования. Задачи компьютерного зрения (Computer Vision, CV) на сегодняшний день являются одними из самых востребованных в области искусственного интеллекта. Автоматическая детекция и классификация объектов находят применение в робототехнике, беспилотном транспорте, системах безопасности и промышленной автоматизации. Распознавание шахматных фигур представляет собой классическую, но сложную задачу для алгоритмов детекции из-за высокого сходства объектов между собой, перекрытий (окклюзий) и зависимости от условий освещения. Решение этой задачи позволяет протестировать и сравнить эффективность современных архитектур нейронных сетей в условиях ограниченных вычислительных ресурсов и специфических наборов данных.

Цель работы заключается в разработке программного модуля для детекции и классификации шахматных фигур на изображении, а также в проведении сравнительного анализа современных архитектур нейронных сетей (YOLO, SSD, Faster R-CNN) для выявления наиболее эффективного метода.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Изучить теоретические основы и архитектурные особенности современных методов детекции объектов (YOLOv11, SSD, Faster R-CNN).
2. Разработать методику и программные средства для генерации синтетического датасета с автоматической разметкой.
3. Сформировать набор реальных данных, обеспечив разнообразие ракурсов и условий освещения.
4. Реализовать обучение моделей, применив методы аугментации данных для повышения их обобщающей способности.
5. Провести сравнительный анализ результатов по метрикам точности (mAP) и скорости инференса.

Объектом исследования являются технологии компьютерного зрения и методы машинного обучения.

Предметом исследования выступают алгоритмы детекции объектов на изображениях применительно к распознаванию шахматных фигур.

В работе использовались методы системного анализа, методы обработки изображений, технологии глубокого обучения (Deep Learning), а также инструменты 3D-моделирования (Blender) и библиотеки компьютерного зрения (OpenCV, PyTorch).

Глава 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ДЕТЕКЦИИ ОБЪЕКТОВ

1.1. Обзор современных методов компьютерного зрения (YOLO, SSD, Faster R-CNN)

В современной практике компьютерного зрения задачи детекции объектов (Object Detection) решаются преимущественно с использованием сверточных нейронных сетей (CNN). Основная цель таких алгоритмов — выделить на изображении ограничивающую рамку (Bounding Box, BB) вокруг объекта и определить его класс. Для реализации курсового проекта были выбраны три популярные архитектуры, различающиеся подходами к обработке изображения.

YOLO (You Only Look Once). Это семейство одностадийных детекторов, которые рассматривают задачу детекции как единую задачу регрессии. Модель делит изображение на сетку и для каждой ячейки предсказывает координаты рамок и вероятности классов.

- *Преимущества:* Высокая скорость работы, возможность использования в реальном времени (Real-Time).
- *Особенности версии YOLOv11:* Это новейшая итерация (SOTA — State of the Art), обладающая оптимизированной архитектурой backbone, что обеспечивает лучший баланс между скоростью и точностью обучения по сравнению с предшественниками. Для работы была выбрана версия Nano (n) как наиболее легковесная.

SSD (Single Shot MultiBox Detector). Также является одностадийным методом. В отличие от ранних версий YOLO, SSD использует карты признаков (feature maps) различного масштаба для детекции объектов разных размеров. Одной из ключевых особенностей является использование заранее заданных рамок (anchor boxes). В данной работе в качестве backbone для SSD была выбрана сеть VGG16.

Faster R-CNN. Представитель двухстадийных детекторов.

1. На первом этапе сеть генерации предложений (Region Proposal Network, RPN) выделяет области, где, вероятно, находятся объекты.

2. На втором этапе эти области классифицируются, и уточняются координаты рамок.

Такой подход традиционно обеспечивает более высокую точность, особенно для мелких объектов, но требует больших вычислительных затрат и времени на обучение. В качестве backbone использовался ResNet50.

1.2. Специфика формирования наборов данных для обучения

Качество работы нейронной сети напрямую зависит от данных. В задачах CV выделяют два типа данных:

1. **Синтетические данные.** Создаются в средах 3D-моделирования. Их преимущество — идеальная автоматическая разметка, отсутствие шумов и возможность сгенерировать неограниченное количество примеров. Недостаток — «синтетический разрыв» (domain gap), когда модель, обученная на 3D-графике, не узнает реальные фото.

2. **Реальные данные.** Фотографии физических объектов. Они содержат естественные шумы, блики, особенности освещения, что критически важно для работы модели в реальных условиях.

В рамках данной работы было принято решение использовать гибридный подход: обучение на большом объеме синтетических данных с дообучением (fine-tuning) или валидацией на реальных фотографиях, а также активное использование методов аугментации (искажение, поворот, изменение яркости) для улучшения обобщающей способности.

Глава 2. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПОДГОТОВКА ДАННЫХ

2.1. Постановка задачи и инструментарий

Задача курсового проекта формулируется как детекция 12 классов объектов (6 типов фигур: король, ферзь, ладья, слон, конь, пешка; двух цветов: белый, черный) на изображении шахматной доски.

- **Входные данные:** Изображения размером 500x500 пикселей.
- **Выходные данные:** Координаты ограничивающих рамок (BB) и метка класса для каждой фигуры.
- **Условия:** Ракурс строго перпендикулярно центру доски (вид сверху). Количество фигур не превышает 12. Фигуры находятся строго в рамках клеток.

Для реализации использовался язык программирования Python и следующие библиотеки:

- **PyTorch** — для построения и обучения нейронных сетей.
- **OpenCV** — для обработки изображений и захвата видеопотока.
- **Blender** — для 3D-моделирования и генерации синтетического датасета (использовался встроенный интерпретатор Python).

2.2. Генерация синтетического набора данных

Для создания обучающей выборки была разработана программа внутри среды Blender. Использовались 3D-модели фигур с сайта Cults3D.

Алгоритм генерации сцены:

1. На виртуальной доске размещаются 12 фигур (уникальных по цвету и форме).
2. Каждая фигура получает случайный поворот (от 0 до 360 градусов) вокруг своей оси.
3. Положение фигур ограничено клетками доски, пересечения исключены.

4. Используются 5 источников освещения (центр и углы), чтобы минимизировать жесткие тени.

Пример получаемого синтетического изображения представлен на рисунке 2.1.

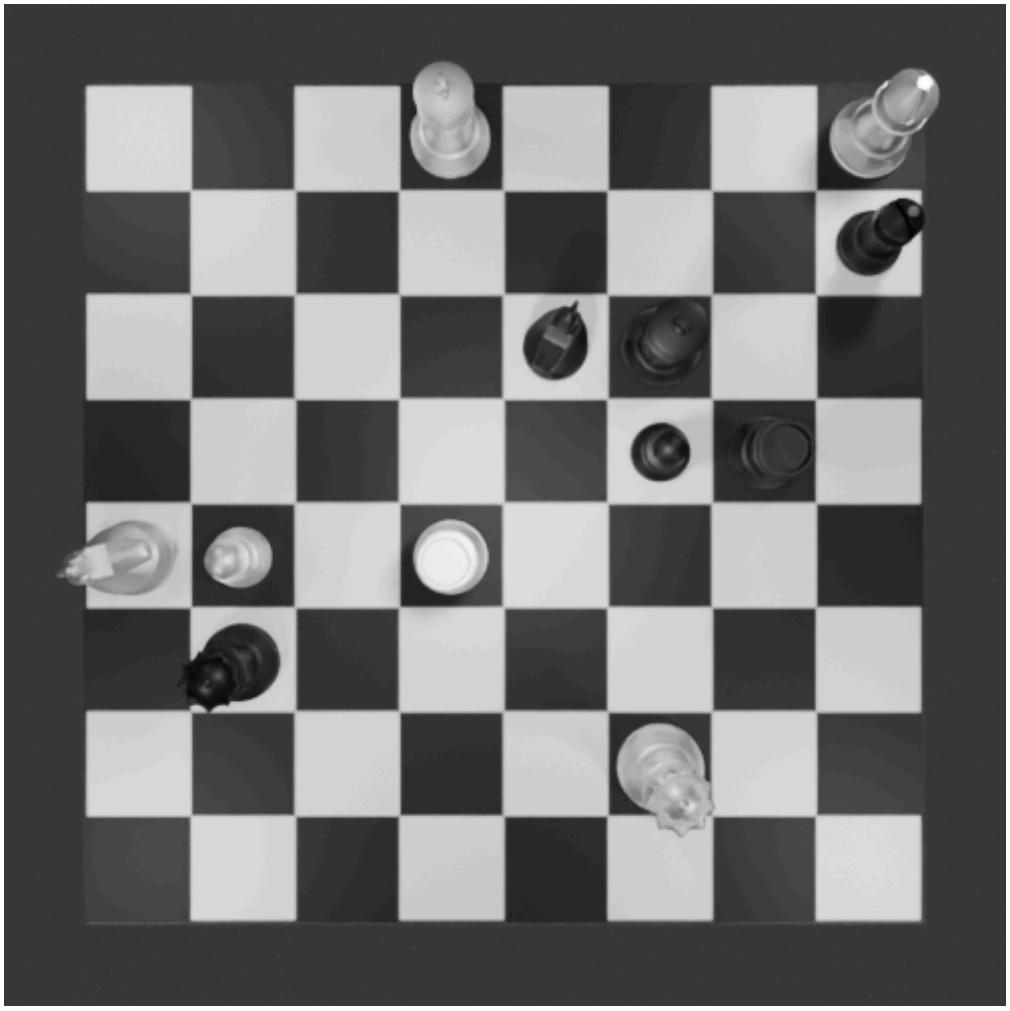


Рисунок 2.1 — Пример синтетического изображения шахматной доски

Автоматическая разметка. Поскольку координаты фигур в 3D-пространстве известны, разметка (Bounding Box) генерировалась программно. Для каждой фигуры создавалась рамка размером 70x70 пикселей вокруг центра клетки. Данные сохранялись в формате CSV, а затем конвертировались в формат TXT (стандарт YOLO).

Всего было сгенерировано 15 000 изображений, что дало 180 000 размеченных объектов. Листинг кода генерации приведен в Приложении А.

2.3. Формирование набора реальных данных

Для проверки работы модели в реальных условиях был создан физический набор данных. Фигуры были распечатаны на 3D-принтере.

Особенности подготовки фигур:

Изначально матовые фигуры сливались с фоном. Для решения проблемы фигуры были покрыты глянцевой краской («Metallic» с высоким коэффициентом отражения). Это позволило создать блики и четкие контуры, различимые для камеры (см. рис. 2.2).

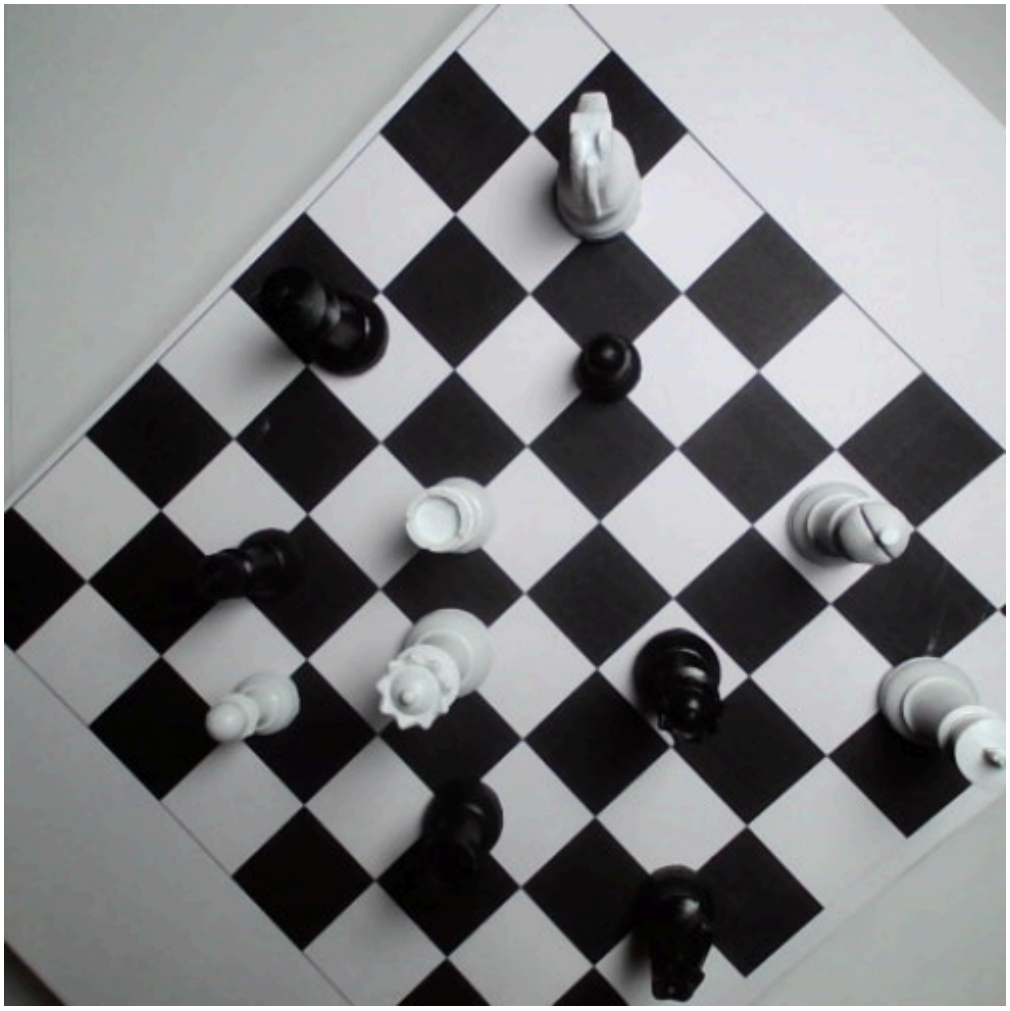


Рисунок 2.2 — Шахматные фигуры после нанесения глянцевого покрытия

Процесс съемки:

Использовалась установка, включающая камеру (разрешение 720p, 30 FPS), прожектор и вращающийся стенд. Съемка велась под тремя углами наклона: ~10°, ~20°, ~30°.

- Видео записывалось в течение полного оборота стенда (24 секунды).
- Общий объем составил 50 000 фотографий.
- Расчет объема данных: 30 кадров/с * 24 с * 12 фигур * 3 наклона * 2 цвета фона (черная/белая клетка).

Для обработки видео был написан скрипт на Python (см. Приложение Б), который автоматически нарезал видео на кадры 500x500 и центрировал изображение так, чтобы фигура всегда находилась в центре. Это позволило использовать фиксированную разметку (BB 70x70 по центру) без необходимости размечать тысячи фото вручную.

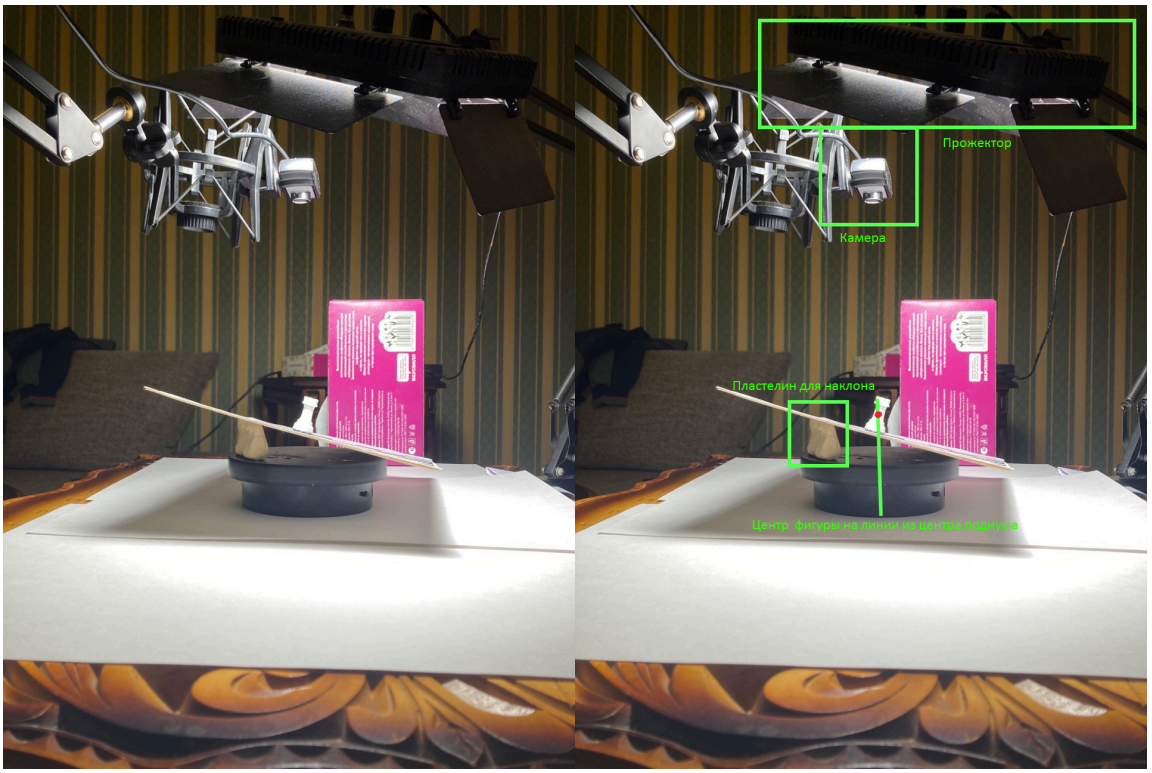


Рисунок 2.3 — Установка для сбора реального датасета

Глава 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ МОДЕЛЕЙ

3.1. Обучение и тестирование модели YOLOv11

В качестве базовой модели была выбрана YOLOv11n (Nano). Исследование проводилось в три этапа.

Этап 1. Обучение только на синтетических данных.

Модель показала идеальную сходимость на обучающей выборке (box_loss снизился до 0.61), однако при тестировании на реальных фотографиях результат оказался неудовлетворительным. Модель не смогла обобщить знания и распознать реальные фигуры из-за отличий в текстурах и освещении.

Этап 2. Внедрение аугментации.

Для решения проблемы низкой обобщающей способности были применены методы аугментации данных:

- *Mosaic:* склеивание 4 случайных изображений в одно. Это ключевая аугментация, позволившая модели учиться распознавать объекты в сложном контексте и разного масштаба.
- *Mixup:* наложение изображений с весами (коэффициент 0.2).
- *Геометрические искажения:* поворот, сдвиг, изменение перспективы.
- *Цветовые коррекции:* изменение Hue, Saturation, Value.

Пример работы аугментации «Mosaic» представлен на рисунке 3.1.

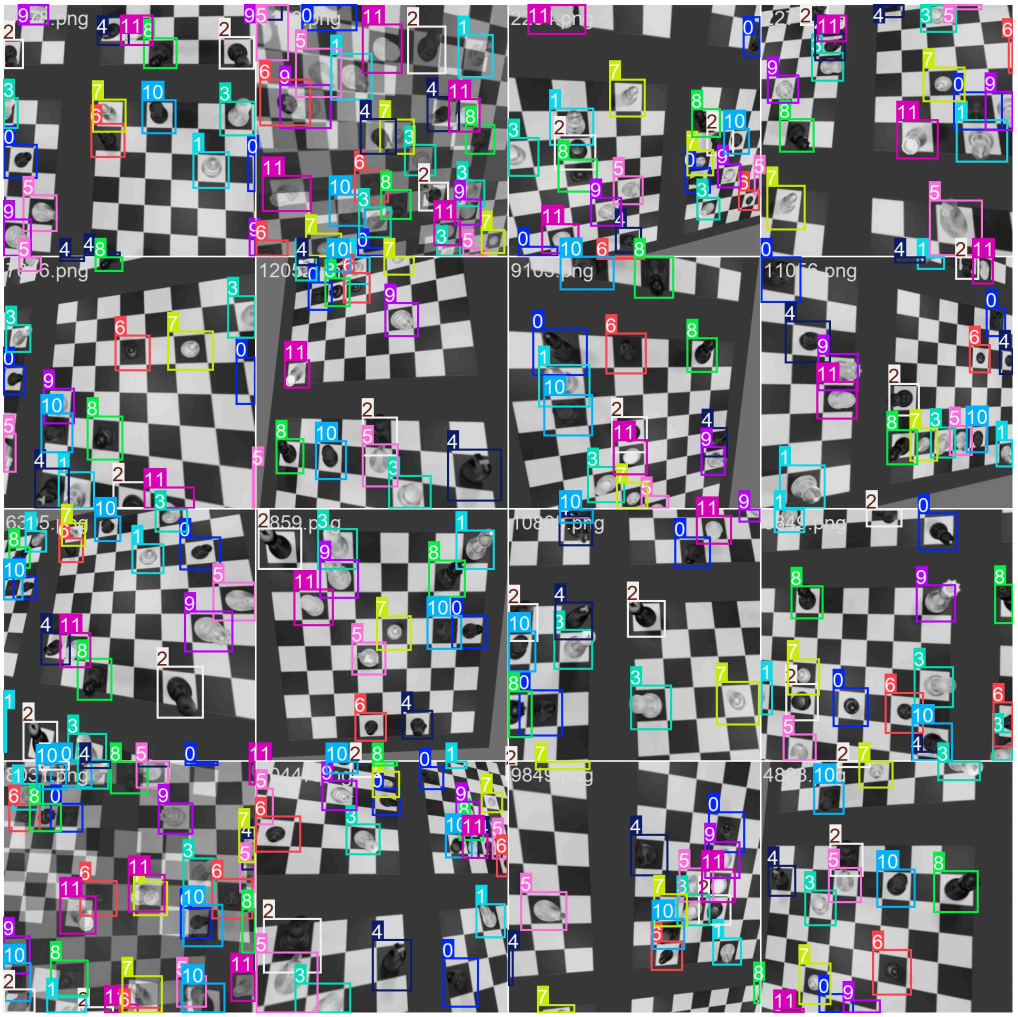


Рисунок 3.1 — Пример аугментации данных методом Mosaic

Этап 3. Обучение на полном датасете.

В итоговый датасет были включены и синтетические, и реальные изображения. Были скорректированы веса функции потерь:

- Вес ошибки классификации (*cls*) увеличен до 5.0 (так как определить класс фигуры сложнее, чем её положение).
- Вес регрессии рамки (*box*) установлен на 1.5.

В результате модель YOLOv11n продемонстрировала высокую точность на тестовой выборке. На рисунке 3.2 представлены графики процесса обучения.

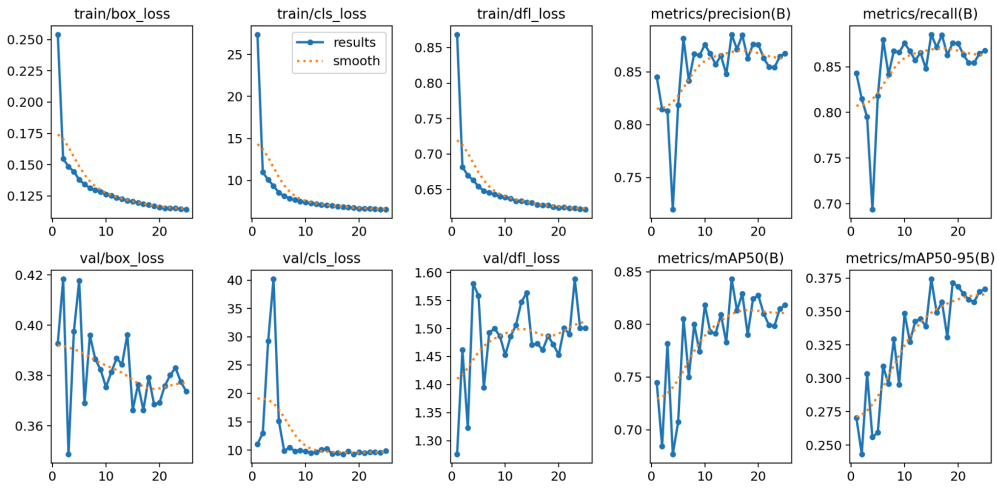


Рисунок 3.2 — Метрики обучения модели YOLOv11n

Примеры детекции на реальных фотографиях показывают, что модель успешно справляется с задачей даже при наличии бликов и теней.



Рисунок 3.3 — Результат детекции фигур моделью YOLOv11n

3.2. Анализ эффективности модели SSD

Для обучения модели SSD использовался backbone VGG16. В ходе экспериментов выявлены следующие особенности:

1. **Скорость обучения:** Время одной эпохи составило 17 минут (против 2-3 минут у YOLO), что существенно замедлило эксперименты.
2. **Сходимость:** Наблюдалась проблема «взрывающихся градиентов» при использовании оптимизатора SGD. Переход на оптимизатор Adam и уменьшение размера батча (batch size = 8) стабилизировали обучение.
3. **Результат:** Модель показала адекватные результаты, но осталась недообученной (согласно PR-кривым и матрице ошибок). Она часто пропускала фигуры, но если находила их — классифицировала верно.

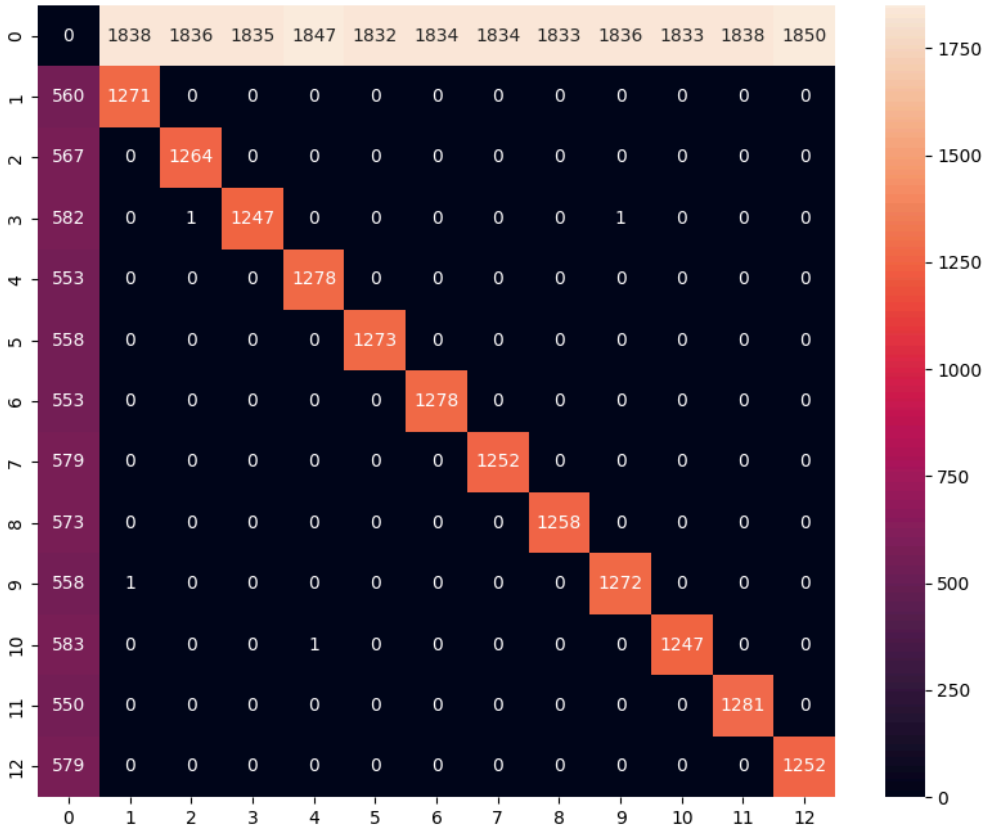


Рисунок 3.4 — Матрица ошибок (Confusion Matrix) для модели SSD

3.3. Исследование архитектуры Faster R-CNN

Модель Faster R-CNN с backbone ResNet50 (98 Мб) является наиболее сложной из рассмотренных.

- **Обучение:** Заняло 8 часов 9 минут на 10 эпох. Из-за большого потребления памяти размер батча был уменьшен до 4.
- **Переобучение:** Из-за высокой емкости модели и недостаточного разнообразия фона на реальных фото (черно-белая клетка) модель быстро переобучилась на тренировочных данных.
- **Точность:** На аугментированных данных модель показала высокую точность, но проиграла YOLO в скорости инференса на несколько порядков, что делает её менее пригодной для задач реального времени.

3.4. Сравнительный анализ результатов

Сводные результаты тестирования трех моделей представлены в таблице 3.1.

Таблица 3.1 — Сравнительные характеристики моделей

Модель	Backbone (Архитектура)	Время обучения	Скорость работы (Inference)	Точность и особенности
YOLOv11n	Custom (Nano)	~3 часа	Высокая (Real-time)	Лучшая обобщающая способность, устойчивость к бликам и шуму.
SSD	VGG16	~4.5 часа	Средняя	Сложности со сходимостью, требует тщательного подбора гиперпараметров.
Faster R-CNN	ResNet50	~8 часов	Низкая	Высокая точность на тестовых данных, но склонность к переобучению и долгий инференс.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно решена задача автоматической детекции и классификации шахматных фигур с использованием методов компьютерного зрения.

Основные результаты работы:

1. **Создан уникальный набор данных.** Сформирован гибридный датасет объемом более 65 000 изображений, включающий как синтетические сцены (Blender), так и реальные фотографии, полученные с помощью разработанной автоматизированной установки.

2. **Проведен сравнительный анализ.** Исследованы архитектуры YOLOv11, SSD и Faster R-CNN. Установлено, что для поставленной задачи наилучшим выбором является **YOLOv11n**. Она показала лучший баланс между точностью и скоростью, а также продемонстрировала высокую устойчивость к изменению ракурса и освещения.

3. **Выявлена роль аугментации.** Экспериментально доказано, что использование агрессивных методов аугментации (Mosaic, Mixup, геометрические искажения) является критически важным этапом обучения. Без них модели не способны корректно работать на реальных данных после обучения на синтетике.

4. **Практическая значимость.** Разработанный программный модуль может быть использован как основа для создания систем автоматической записи шахматных партий (цифровой протокол) или для управления роботизированным манипулятором, играющим в шахматы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федеральный закон от 29.12.2012 № 273-ФЗ «Об образовании в Российской Федерации».
2. Jocher G. YOLOv11: An Overview of the Key Architectural Enhancements [Электронный ресурс]. URL: https://docs.ultralytics.com/models/yolo11/ (дата обращения: 10.05.2025).
3. Ren S., He K., Girshick R., Sun J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2017. — Vol. 39, No. 6. — P. 1137–1149.
4. Liu W., Anguelov D., Erhan D., Szegedy C. SSD: Single Shot MultiBox Detector // European Conference on Computer Vision (ECCV). — Springer, Cham, 2016. — P. 21–37.
5. Документация библиотеки PyTorch [Электронный ресурс]. URL: https://pytorch.org/docs/stable/index.html (дата обращения: 15.05.2025).
6. Redmon J., Farhadi A. YOLOv3: An Incremental Improvement [Электронный ресурс] // arXiv preprint arXiv:1804.02767. — 2018. URL: https://arxiv.org/abs/1804.02767.
7. Официальная документация Blender Python API [Электронный ресурс]. URL: https://docs.blender.org/api/current/ (дата обращения: 12.05.2025).

ПРИЛОЖЕНИЕ А

Код генерации синтетических данных

```
import bpy
import random
import math
import os
import numpy as np

num_samples = 10000

model_dir = "C:/Program Files (x86)/Steam/steamapps/common/Blender/Project"
board_file = os.path.join(model_dir, "chess_board.obj")
piece_files = {
    "bishop_black": "bishop_black.obj",
    "bishop_white": "bishop_white.obj",
    "king_black": "king_black.obj",
    "king_white": "king_white.obj",
    "knight_black": "knight_black.obj",
    "knight_white": "knight_white.obj",
    "pawn_black": "pawn_black.obj",
    "pawn_white": "pawn_white.obj",
    "queen_black": "queen_black.obj",
    "queen_white": "queen_white.obj",
    "rook_black": "rook_black.obj",
    "rook_white": "rook_white.obj",
}

output_dir = "C:/Users/jack_/MAIN/ВУЗ/Курсовая/Dataset/Ver1"
os.makedirs(output_dir, exist_ok=True)
target_file = os.path.join(output_dir, "targets.csv")

# Очистка сцены
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete()

# Импортируем доску
```

```
bpy.ops.wm.obj_import(filepath=board_file)
board = bpy.context.selected_objects[0]
board.name = "Chessboard"

# Список всех фигур
pieces = {}

for piece_name, file_name in piece_files.items():
    bpy.ops.wm.obj_import(filepath=os.path.join(model_dir, file_name))
    piece = bpy.context.selected_objects[0]
    piece.name = piece_name
    piece.location = (100, 100, 0)  # Уводим за пределы видимости
    pieces[piece_name] = piece

bpy.ops.object.camera_add(location=(0, 0, 35))
cam = bpy.context.object
cam.rotation_euler = (0, 0, 0)
cam.data.type = 'PERSP'
bpy.context.scene.camera = cam

bpy.context.scene.render.resolution_x = 500
bpy.context.scene.render.resolution_y = 500
bpy.context.scene.render.resolution_percentage = 100


# Добавляем источник света
bpy.ops.object.light_add(type='SUN', location=(0, 0, 8))
light = bpy.context.object
light.rotation_euler = (0, 0, 0)

bpy.ops.object.light_add(type='AREA', location=(0, 0, 10))
bpy.context.object.data.energy = 1000
bpy.ops.object.light_add(type='AREA', location=(-10, -10, 10))
bpy.context.object.data.energy = 1000
bpy.ops.object.light_add(type='AREA', location=(+10, -10, 10))
bpy.context.object.data.energy = 1000
bpy.ops.object.light_add(type='AREA', location=(+10, +10, 10))
```

```
bpy.context.object.data.energy = 1000
bpy.ops.object.light_add(type='AREA', location=(-10, +10, 10))
bpy.context.object.data.energy = 1000



# Включаем рендеринг текстур и освещения
bpy.context.scene.render.engine = 'CYCLES'
#bpy.context.scene.render.engine = 'BLENDER_EEVEE'
bpy.context.scene.render.image_settings.file_format = 'PNG'
prefs = bpy.context.preferences.addons['cycles'].preferences
prefs.compute_device_type = 'CUDA'  # или 'OPTIX' / 'HIP' / 'METAL' — зависит от видеокарты
bpy.context.scene.cycles.device = 'GPU'
bpy.context.scene.cycles.samples = 32

# Получаем начальный номер итерации
start_id = 0
if os.path.exists(target_file):
    with open(target_file, "r") as f:
        lines = f.readlines()
        if len(lines) > 1:
            last_line = lines[-1]
            last_id = int(last_line.split(",")[0])
            start_id = last_id + 1

# Открываем файл в режиме дозаписи
with open(target_file, "a" if start_id > 0 else "w") as f:
    if start_id == 0:
        f.write("image_id,piece,x,y,rot_z\n")

    for i in range(start_id, start_id + num_samples):
        placed_pieces = []
        occupied_cells = set()

        for piece_name, piece in pieces.items():
            while True:
                place_x = np.random.randint(0, 8)
                place_y = np.random.randint(0, 8)
```

```
            if (place_x, place_y) not in occupied_cells:
                occupied_cells.add((place_x, place_y))
                break

        x = -8.96654 + place_x * 2.5474
        y = -8.91536 + place_y * 2.5474
        rot_z = random.uniform(0, 2 * math.pi)

        piece.location = (x, y, 1)
        piece.rotation_euler = (math.pi/2, 0, rot_z)

        placed_pieces.append((piece_name, place_x, place_y, rot_z))

    image_path = os.path.join(output_dir, f"{i}.png")
    bpy.context.scene.render.filepath = image_path
    bpy.ops.render.render(write_still=True)

    for piece_data in placed_pieces:
        f.write(f"{i},{piece_data[0]},{piece_data[1]},{piece_data[2]},{piece_data[3]}\n")
data_path = r"D:\ВУЗ\Курсовая работа 2025\Dataset\ver3"
out_path = r"D:\ВУЗ\Курсовая работа 2025\Код + Dataset ver3"
csv_path = os.path.join(data_path, "targets.csv")
output_path = os.path.join(out_path, "datasets")
test_percent = 0.1

# Параметры доски
offset = 40          # отступ от края до доски
cell_size = 52       # размер клетки
img_size = 500       # размер изображения
board_size = 8

# Параметры bounding box
bb_w = 70            # ширина bb
bb_h = 70            # высота bb

# Чтение targets.csv
df = pd.read_csv(csv_path, header=None, names=["file", "label", "x", "y", "angle"], skiprows=1)
```

```
for i in ["x", "y"]:
    df[i] = df[i].apply(lambda k: np.int64(k))

# Список классов
labels = sorted(df["label"].unique())
label2id = {label: i for i, label in enumerate(labels)}

# Создание папок
for split in ["train", "test"]:
    os.makedirs(os.path.join(output_path, "images", split), exist_ok=True)
    os.makedirs(os.path.join(output_path, "labels", split), exist_ok=True)

# Разделение на train/test
all_files = sorted({f"{row}.png" for row in df["file"]})
random.shuffle(all_files)
split_idx = int(len(all_files) * (1 - test_percent))
train_files = set(all_files[:split_idx])
test_files = set(all_files[split_idx:])

# Генерация аннотаций
grouped = df.groupby("file")
for file_name, group in grouped:
    txt_lines = []
    for _, row in group.iterrows():
        cx = offset + (row["x"] + 0.5) * cell_size
        cy = img_size - (offset + (row["y"] + 0.5) * cell_size)
        yolo_x = cx / img_size
        yolo_y = cy / img_size
        yolo_w = bb_w / img_size
        yolo_h = bb_h / img_size
        class_id = label2id[row["label"]]
        txt_lines.append(f"{class_id} {yolo_x:.6f} {yolo_y:.6f} {yolo_w:.6f} {yolo_h:.6f}")

    split = "train" if f"{file_name}.png" in train_files else "test"
    with open(os.path.join(output_path, "labels", split, f"{file_name}.txt"), "w") as f:
        f.write("\n".join(txt_lines))
```

```
    shutil.copy(
        os.path.join(data_path, f"{file_name}.png"),
        os.path.join(output_path, "images", split, f"{file_name}.png")
    )

# Классы
with open(os.path.join(output_path, "classes.txt"), "w") as f:
    for label in labels:
        f.write(f"{label}\n")
```

ПРИЛОЖЕНИЕ Б

Код захвата и разметки реальных данных

```
cap = cv2.VideoCapture(1)
hw = 35
max_frames = 720
i = 0
record = False
piece = 'pawn'
colour = 'black'
checkerboard = 'white'
angle = '0'
name = f'{piece}_{colour}'



fig_names = '''bishop_black
bishop_white
king_black
king_white
knight_black
knight_white
pawn_black
pawn_white
queen_black
queen_white
rook_black
rook_white'''.split()
d = {fig_names[i]:i for i in range(len(fig_names))}



# Папки
root = f'TEST_real_set/{name}'
img_dir = os.path.join(root, 'images')
lbl_dir = os.path.join(root, 'labels')
os.makedirs(img_dir, exist_ok=True)
os.makedirs(lbl_dir, exist_ok=True)
```

```
print("Нажми 's' для старта записи, 'q' для выхода")

while True:
    ret, frame = cap.read()
    if not ret: break

    h, w, _ = frame.shape
    size = min(h, w)
    cx, cy = w//2, h//2
    x1, y1 = cx - size//2, cy - size//2
    x2, y2 = cx + size//2, cy + size//2

    crop = frame[y1:y2, x1:x2]  # центрированный квадрат
    f = cv2.resize(crop, (500, 500))

    bb_cx, bb_cy = 0.5, 0.5
    bb_w = bb_h = 70 / 500

    # превью
    p = f.copy()
    r1, r2 = 250 - hw, 250 + hw
    cv2.rectangle(p, (r1, r1), (r2, r2), (0,0,255), 1)
    cv2.imshow('setup', p)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('s'):
        print("Запись начата...")
        record = True
    if key == ord('q'):
        break

    if record and i < max_frames:
        fn = f'{name}_{i:04d}_{checkerboard}_angle_{angle}'
        cv2.imwrite(f'{img_dir}/{fn}.png', f)
        with open(f'{lbl_dir}/{fn}.txt', 'w') as L:
            L.write(f"{d[name]} {bb_cx} {bb_cy} {bb_w:.6f} {bb_h:.6f}")
        i += 1
```

```
        if i == max_frames:
            print("Максимум кадров достигнут.")
            break

cap.release()
cv2.destroyAllWindows()
```